Отчёт по лабораторной работе 8

Оптимизация

Цвелев С.А. НПИбд-01-22

Содержание

1 Цель работы

Основная цель работы — освоить пакеты Julia для решения задач оптимизации.

2 Ход работы

8.4.1. Линейное программирование

Решите задачу линейного программирования:

$x_1 + 2x_2 + 5x_3 \to \max$,

при заданных ограничениях:

$-x_1 + x_2 + 3x_3 \le -5$,

$x_1 + 3x_2 - 7x_3 \le 10$,

$0 \le x_1 \le 10$,

$x_2 \ge 0$,

$x_3 \ge 0$.

[3]:
```
model = Model(GLPK.Optimizer)

@variable(model, 0 <= x1 <= 10)
@variable(model,x2 >= 0)
@variable(model, x3 >= 0)

@constraint(model, -x1 + x2 + x3 <= -5)
@constraint(model, x1 + 3x2 - 7x3 <= 10)

@objective(model, Max, x1 + 2x2 + 5x3)

optimize!(model)

println("x1 = ", value(x1), " ", "x2 = ", value(x2), " ", "x3 = ", value(x3))
```

```
x1 = 10.0 x2 = 0.0 x3 = 5.0
```

8.4.2. Линейное программирование. Использование массивов

Решите предыдущее задание, используя массивы вместо скалярных переменных.

Рекомендация. Запишите систему ограничений в виде $A \cdot x = b$, а целевую функцию как $c^T x$.

```
[5]: A = [-1 1 3; 1 3 -7]
     b = [-5; 10]
     c = [1; 2; 5]

     model = Model(GLPK.Optimizer)

     @variable(model, x[1:3] >= 0)
     @constraint(model, x[1] <= 10)
     @constraint(model, A * x .<=b)
     @objective(model, Max, c' * x)

     optimize!(model)

     println("x = ", value.(x))

     x = [10.0, 2.1875, 0.9375]
```

8.4.3. Выпуклое программирование

Решите задачу оптимизации:

$\|A \cdot x - b\|_2^2 \to \min$

при заданных ограничениях:

$x \geq 0$,

где $x \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$.

Матрицу A и вектор b задайте случайным образом.

Для решения задачи используйте пакет Convex и решатель SCS.

```
[6]: Random.seed!(0)

m, n = 30, 10
A = randn(m, n)
b = randn(m)

x = Convex.Variable(n)

problem = minimize(sumsquares(A * x - b), [x >= 0])
solve!(problem, SCS.Optimizer)

println("status = ", problem.status)
println("x = ", vec(x.value))
```

```
------------------------------------------------------------------
               SCS v3.2.11 - Splitting Conic Solver
        (c) Brendan O'Donoghue, Stanford University, 2012
------------------------------------------------------------------
problem:  variables n: 11, constraints m: 42
cones:    l: linear vars: 10
          q: soc vars: 32, qsize: 1
settings: eps_abs: 1.0e-04, eps_rel: 1.0e-04, eps_infeas: 1.0e-07
          alpha: 1.50, scale: 1.00e-01, adaptive_scale: 1
          max_iters: 100000, normalize: 1, rho_x: 1.00e-06
          acceleration_lookback: 10, acceleration_interval: 10
          compiled with openmp parallelization enabled
lin-sys:  sparse-direct-amd-qdldl
          nnz(A): 312, nnz(P): 0
------------------------------------------------------------------
 iter | pri res | dua res |   gap   |   obj   |  scale  | time (s)
------------------------------------------------------------------
     0| 6.19e+01  1.00e+00  8.76e+01 -4.28e+01  1.00e-01  2.07e-03
   250| 4.58e-04  4.81e-05  5.89e-04  2.47e+01  4.30e-01  6.43e-03
------------------------------------------------------------------
status:  solved
timings: total: 6.43e-03s = setup: 1.43e-04s + solve: 6.29e-03s
         lin-sys: 2.30e-04s, cones: 3.89e-05s, accel: 1.44e-03s
------------------------------------------------------------------
objective = 24.721028
------------------------------------------------------------------
status =
[ Info: [Convex.jl] Compilation finished: 4.64 seconds, 663.955 MiB of memory allocated
OPTIMAL
x = [0.000123658929598414 94, -1.599703544438304 13e-5, 0.000458021537585501, 0.230133801346822 24, 0.145182733315878613, 2.039003979958700 6e-5, 0.195914809793846 23, 0.009438689011214816, 0.264387287915833 9, 0.000236638550113747 37]
```

8.4.4. Оптимальная рассадка по залам

Проводится конференция с 5 разными секциями. Забронировано 5 залов различной вместимости: в каждом зале не должно быть меньше 180 и больше 250 человек, а на третьей секции активность подразумевает, что должно быть точно 220 человек.

В заявке участник указывает приоритет посещения секции: 1 — максимальный приоритет, 3 — минимальный, а значение 10000 означает, что человек не пойдёт на эту секцию.

Организаторам удалось собрать 1000 заявок с указанием приоритета посещения трёх секций. Необходимо дать рекомендацию слушателю, на какую же секцию ему пойти, чтобы хватило места всем.

Для решения задачи используйте пакет Convex и решатель GLPK.

Приоритеты по слушателям распределите случайным образом.

8.4.5. План приготовления кофе

Кофейня готовит два вида кофе: «Раф кофе» за 400 рублей и «Капучино» за 300 рублей.

Чтобы сварить 1 чашку «Раф кофе», необходимо: 40 г зёрен, 140 г молока и 5 г ванильного сахара.

Чтобы приготовить 1 чашку «Капучино», необходимо: 30 г зёрен и 120 г молока.

На складе есть: 500 г зёрен, 2000 г молока и 40 г ванильного сахара.

Необходимо найти план варки кофе, обеспечивающий максимальную выручку от их реализации. При этом необходимо потратить весь ванильный сахар.

Для решения задачи используйте пакет JuMP и решатель GLPK.

```
[7]: model = Model(GLPK.Optimizer)

@variable(model, raf >= 0, Int)
@variable(model, cap >= 0, Int)

@constraint(model, 40raf + 30cap <= 500)
@constraint(model, 140raf + 120cap <= 2000)
@constraint(model, 5raf == 40)

@objective(model, Max, 400raf + 300cap)

optimize!(model)

println("Раф = ", value(raf), " ", "Капучино = ", value(cap))
```

```
Раф = 8.0 Капучино = 6.0
```

3 Вывод

Мы освоили пакеты Julia для решения задач оптимизации.